TASK 18 — MISSING VALUES IN PANDAS

Data Science Track

Description: Identify missing values in a dataset and practice handling them using dropping and filling techniques.

Objective: Understand one of the most common data preprocessing problems.

1. Tools

• Python
• Pandas
• Jupyter Notebook

2. Deliverables

• Missing-value summary
• Examples using isna() and notna()
• A cleaned version of the dataset

3. Missing Values — Meaning

A missing value is a field for which no usable value is available. It can result from unavailable information, skipped data entry, measurement problems, or data-transfer issues.

4. Why Missing Values Are a Problem

Missing data can affect averages, counts, visualizations, reports and machine-learning models. Before analysis, the analyst should measure missingness and decide how to handle it rather than ignoring it.

5. Task Workflow

The efficient workflow is: **detect** → **summarize** → **investigate** → **choose strategy** → **clean** → **validate** → **save.** This avoids blindly modifying the dataset.

6. isna() — Detect Missing Values

isna() returns True where a value is missing and False where a value is present.

```
df.isna()
```

7. notna() — Detect Present Values

notna() is the opposite of isna(). It returns True where the value is available.

```
df.notna()
```

8. Count Missing Values

Use isna().sum() to count missing values column by column. This is the required missing-value summary for the task.

```
df.isna().sum()
```

9. Total Missing Values

Apply sum() again to obtain the total number of missing cells.

```
df.isna().sum().sum()
```

10. Find Rows with Missing Values

any(axis=1) checks each row. If any column in that row is missing, the row is returned.

```
df[df.isna().any(axis=1)]
```

11. Find Columns with Missing Values

This identifies which column names contain at least one missing value.

```
df.columns[df.isna().any()].tolist()
```

12. dropna() — Removing Missing Records

dropna() removes rows containing missing values by default. It is simple and useful when only a small number of rows are incomplete.

```
df.dropna()
```

13. When Should Rows Be Dropped?

Drop rows when incomplete records are few, the missing fields are critical, or the records cannot be reliably recovered. Avoid it when many rows are missing because the resulting dataset may become too small or biased.

14. Dropping Columns

axis=1 makes dropna() operate on columns. This should be used carefully because removing an important feature can reduce the usefulness of the dataset.

```
df.dropna(axis=1)
```

15. fillna() — Filling Missing Values

fillna() replaces missing values with a chosen value. This is called imputation and allows the record to remain in the dataset.

```
df["Marks"] = df["Marks"].fillna(value)
```

16. Mean Imputation

For a numerical column, replace missing values with the column mean. This is easy but the mean can be distorted by extreme values.

```
df["Python_Marks"] = df["Python_Marks"].fillna(df["Python_Marks"].mean())
```

17. Median Imputation

The median is the middle value after sorting. It is generally more resistant to outliers and is often a safer simple choice for skewed numerical data.

```
df["Python_Marks"] = df["Python_Marks"].fillna(df["Python_Marks"].median())
```

18. Mode Imputation

Mode is the most frequently occurring value. It is commonly used for categorical columns such as Department, Status or Category.

```
df["Status"] = df["Status"].fillna(df["Status"].mode()[0])
```

19. Mean vs Median vs Mode

Mean: numerical average; sensitive to outliers.
Median: middle numerical value; more robust to outliers.
Mode: most frequent value; suitable for categories.

Method	Best suited for	Main caution
Mean	Balanced numerical data	Outliers can distort it
Median	Skewed/outlier-prone numerical data	May ignore distribution shape
Mode	Categorical data	Can overrepresent the most common category
dropna()	Small amount of missing/incomplete records	Can lose useful data

20. Recommended Strategy for This Project

For this student dataset, the efficient final approach is **median for numerical columns** and **mode for categorical columns**. The project also demonstrates mean and row-dropping so the strategies can be compared.

```
cleaned = df.copy()

for col in cleaned.select_dtypes(include="number").columns:
    cleaned[col] = cleaned[col].fillna(cleaned[col].median())

for col in cleaned.select_dtypes(include="object").columns:
    if cleaned[col].isna().any():
        cleaned[col] = cleaned[col].fillna(cleaned[col].mode()[0])
```

21. Validate the Cleaning

Never assume the cleaning worked. Run a second missing-value check. A zero count means no missing values remain in the checked columns.

```
cleaned.isna().sum()
```

22. Complete Efficient Program

This program covers all required deliverables and keeps the workflow organized into detection, comparison, cleaning and validation.

```
import pandas as pd

# Load the dataset
df = pd.read_csv("student_missing_values_raw.csv")

# 1. Missing-value summary
missing_summary = df.isna().sum()
print("=== MISSING VALUES BY COLUMN ===")
```

```
print(missing_summary)

print("\nTotal missing values:", df.isna().sum().sum())

# 2. Missing and non-missing checks
print("\n=== ISNA ===")
print(df.isna())

print("\n=== NOTNA ===")
print(df.notna())

# 3. Rows and columns affected
print("\n=== ROWS WITH MISSING VALUES ===")
print(df[df.isna().any(axis=1)])

print("\n=== COLUMNS WITH MISSING VALUES ===")
print(df.columns[df.isna().any()].tolist())

# 4. Drop incomplete rows
drop_rows = df.dropna()
print("\n=== AFTER DROPNA ===")
print(drop_rows)

# 5. Mean imputation for numerical columns
mean_df = df.copy()
for col in mean_df.select_dtypes(include="number").columns:
    mean_df[col] = mean_df[col].fillna(mean_df[col].mean())

# 6. Median imputation for numerical columns
median_df = df.copy()
for col in median_df.select_dtypes(include="number").columns:
    median_df[col] = median_df[col].fillna(median_df[col].median())

# 7. Mode imputation for categorical columns
mode_df = df.copy()
for col in mode_df.select_dtypes(include="object").columns:
    if mode_df[col].isna().any():
        mode_df[col] = mode_df[col].fillna(mode_df[col].mode()[0])

# 8. Final practical cleaning
cleaned = df.copy()

numeric_cols = cleaned.select_dtypes(include="number").columns
categorical_cols = cleaned.select_dtypes(include="object").columns

for col in numeric_cols:
    cleaned[col] = cleaned[col].fillna(cleaned[col].median())

for col in categorical_cols:
    if cleaned[col].isna().any():
        cleaned[col] = cleaned[col].fillna(cleaned[col].mode()[0])

print("\n=== CLEANED DATA ===")
print(cleaned)

print("\n=== REMAINING MISSING VALUES ===")
print(cleaned.isna().sum())

# Save cleaned data
cleaned.to_csv("student_missing_values_cleaned.csv", index=False)
```

23. Practical Interpretation

The raw dataset intentionally contains missing marks, attendance, project and status values. The cleaned dataset keeps all student records while replacing missing numerical values with column medians and missing categorical values with their modes. This is a simple, explainable preprocessing workflow.

24. Real-World Applications

- Education: incomplete marks and attendance.
- HR: missing salary or department information.
- Sales: incomplete order attributes.
- Healthcare: missing measurements.
- E-commerce: missing price, rating or product attributes.

25. Common Mistakes

- Using mean for categories.
- Dropping rows without measuring data loss.
- Using one strategy for every column.
- Forgetting to validate after filling.
- Replacing missing data without understanding the column.

26. Best Practices

• Profile missingness before cleaning.
• Preserve the raw dataset.
• Choose a strategy based on data type and distribution.
• Document the chosen method.
• Validate the cleaned output.
• For serious projects, investigate why values are missing before deciding how to impute them.

27. Interview Questions and Answers

Q1. What are missing values? Values that are absent or unavailable for a field.

Q2. How do you count them? df.isna().sum().

Q3. What is isna()? It detects missing values.

Q4. What is notna()? It detects values that are present.

Q5. What does dropna() do? It removes rows or columns containing missing values depending on the axis/settings.

Q6. What does fillna() do? It replaces missing values with a chosen value or strategy.

Q7. When is median preferable to mean? When numerical data is skewed or affected by outliers.

Q8. Why not always drop missing rows? Because useful records may be lost and the dataset may become biased or too small.

28. Quick Revision

Detect: df.isna()
Count: df.isna().sum()
Total: df.isna().sum().sum()
Present: df.notna()
Rows: df[df.isna().any(axis=1)]
Drop: df.dropna()
Fill: df.fillna(value)
Mean: df['col'].fillna(df['col'].mean())
Median: df['col'].fillna(df['col'].median())
Mode: df['col'].fillna(df['col'].mode()[0])

29. Top 5 Skills

1. Pandas
2. Data Cleaning
3. Missing-Value Handling
4. Data Preprocessing
5. Python Data Analysis

30. Feedback

The task was very useful for understanding how missing values affect data quality. I learned how to detect missing values with isna() and notna(), remove incomplete rows with dropna(), and fill values using mean, median and mode. The practical dataset improved my understanding of data cleaning and preprocessing.